Checking P4 Parsers and Tables against Grammatical Specifications

Baber Rehman
Huawei Technologies, Hong Kong SAR, China

Abstract

P4 programs describe which packet headers a switch or NIC may parse and in which order match-action tables run. Those two facts are architectural: they are stable, small, and independent of the bit tests that select one header or the control-plane entries that fill a table. We develop a grammatical specification language for both facts, following a proposal that programmers write the intended header sequences and table pipeline in a BNF-like notation and that the compiler reject an implementation trace outside that notation. We give the notation a trace semantics. Sequential composition and alternation build finite languages of header words and of hit/miss events. The miss-guard, written with an arrow, admits its continuation only after a miss, which is the reading forced by the original counterexample. Conformance is language inclusion. On an acyclic fragment of parsers and apply blocks, inclusion is decided by enumerating paths, treating every `select` arm as feasible. Acceptance implies that every concrete trace in the fragment lies in the specification, because the enumerated language over-approximates the packet language. The same semantics yields a parser skeleton whose nondeterministic language equals the grammar; discriminators for alternation remain holes, because the grammar does not contain them. The development is pen-and-paper. It is not a p4c pass and it reports no measurements. The case studies are hand executions of the examples that motivated the notation.

2012 ACM Subject Classification Networks → Programmable networks; Software and its engineering → Formal methods; Theory of computation → Grammars and context-free languages

Keywords and phrases P4, programmable data planes, grammatical specifications, static analysis

Related Version *Workshop note*: `https://doi.org/10.1145/3750022.3750458`

1 Introduction

Programmable data planes, written in P4 [2, 14], move protocol design into the same artifact as the forwarding program. A parser is a state machine that extracts headers. A control block applies match-action tables, and the hit or miss of one table selects the next. Both pieces are ordinary programs, and both have been the subject of substantial verification work: assertion checkers backed by solvers [11, 16, 8], type systems for header layout [6, 7], certified parser equivalence [5], and a definitional semantics [4]. Those tools answer questions about values, safety, and equivalence. They are heavier than the question a programmer often wants answered first: does this parser accept only the header sequences I wrote down, and do these tables run only in the order I wrote down?

1.1 A parser the compiler accepts

Suppose a switch accepts four header sequences. Every packet starts with Ethernet, continues with one IP version, and ends with one transport header. TCP never sits directly on Ethernet. The two columns are the same parser. The left column is ordinary P4, and the reference compiler accepts it [13]. `extract` is applied to a header, and `parse_tcp` is reachable, so neither the type checker nor `SimplifyParsers` has a reason to complain. The right column is the same program with a grammatical specification attached, in the form of the workshop note [17]. The states do not change. The specification is a comment on the parser.

P4

```
state start {
  pkt.extract(hdr.ethernet);
  transition select(
    hdr.ethernet.etherType) {
    0x0800:  parse_ipv4;
    default:  parse_tcp;
  }
}
state parse_tcp {
  pkt.extract(hdr.tcp);
  transition accept;
}
```

P4 with a specification

```
// { TopParser ::= Ether & (IPV4 | IPV6) & (TCP | UDP) }
state start {
  pkt.extract(hdr.ethernet);
  // ok
  transition select(hdr.ethernet.etherType) {
    0x0800:  parse_ipv4;
    default:  parse_tcp;
  }
}
state parse_tcp {
  pkt.extract(hdr.tcp);
  // not ok:  TCP after Ether
  transition accept;
}
```

On the default arm the extracted word is *Ether TCP*. The attached specification allows Ethernet, then one IP version, then TCP or UDP, and that word is not among them. A later control block can still read `hdr.tcp` safely in the P4 sense: the header was extracted, so it is valid, and p4c's invalid-header warning stays silent. The bug is the sequence, not a use of an invalid header. Assertion checkers can be told about the sequence [11, 16, 8], but only by writing a predicate at a program point and discharging it with a solver or a symbolic executor. Nothing in the program on the left states the four legal words.

The model in this paper takes the grammar as the specification. Conformance is inclusion of the parser's path language in the grammar. The default arm is kept, because every `select` arm is treated as feasible, and inclusion then fails. A parser that extracts TCP only after IPv4 or IPv6 conforms. The same discipline applies to tables. If the specification says a fallback table may run only when an earlier table misses, an apply block that runs it on a hit is a well-typed P4 program and a trace outside the grammar. Section 8 checks both examples by hand. Section 9 records why the compiler does not.

A short workshop note proposed to write this check as a grammar attached to the program [17]. The programmer attaches a grammar to a parser and a second grammar to an apply block. Header names are terminals. Sub-parsers are nonterminals. Tables are terminals of a different kind, and three operators record patterns that show up in public P4 programs: one table after another, a fallback table on a miss, and a choice between two tables. The compiler is supposed to reject an implementation path that the grammar does not allow, including a read of two headers that the grammar says cannot be valid together. The note cites correctness-by-construction [3, 12, 10] and leaves an implementation in the reference compiler [13] as future work. It does not define the operators, and its examples are checks of an existing program rather than derivations of one.

This paper supplies the missing definitions and states what a checker for them would be entitled to claim. The contribution is a specification language and a conformance relation, not a tool and not an evaluation.

- We define header grammars and table grammars whose languages are finite sets of words. The table language is a set of hit/miss traces. The miss-guard admits a continuation only on a miss. Inclusion of implementation traces in that language is the conformance relation suggested by the original counterexamples, which reject extra paths and do not reject missing ones.
- We extract a path language from an acyclic fragment of P4 parsers and apply blocks. Every `select` arm is kept. The extracted language therefore over-approximates the traces a packet can produce, and inclusion of the over-approximation implies inclusion of the packet language.
- We show that the same grammars compile to parser skeletons with holes where a P4 `select` must still be written. The nondeterministic skeleton matches the grammar exactly.

Filling a hole can only remove traces.

- We place the design next to solver-based verifiers, SafeP4-style type systems, and certified parser equivalence, and we list the P4 constructs the fragment refuses rather than silently ignores.

Section 2 walks through the examples. Sections 3–7 give the languages, the extraction, the inclusion check, header validity, and skeletons. Section 8 executes the examples by hand. Section 9 compares those decisions with the checks in the reference compiler. Section 10 compares related systems. Section 11 records what is not claimed.

2 Overview

A parser specification is an expression in header names. The expression

$$\mathit{Ether}\ \&\ (\mathit{IPV4} \mid \mathit{IPV6})\ \&\ (\mathit{TCP} \mid \mathit{UDP})$$

denotes four words. Each word starts with Ethernet, continues with one IP version, and ends with one transport header. The corresponding P4 parser is a state machine whose transitions extract those headers. One faulty transition extracts `TCP` in a state that has only extracted `Ether`. The word *Ether TCP* is not in the grammar, so the parser does not conform.

The same idea factors through sub-parsers. A nonterminal *L3* bound to $\mathit{IPV4} \mid \mathit{IPV6}$ is a name for that alternation. The top-level specification $\mathit{L2}\ \&\ \mathit{L3}\ \&\ \mathit{L4}$ is the concatenation of the three languages. A parser that calls the three sub-parsers in that order conforms. A parser that calls *L4* before *L3* does not.

Header validity falls out of the same four words. No word contains both *IPV4* and *IPV6*. A control block that reads a field of each header, with no intervening test, assumes a packet the grammar has already excluded.

Tables use a different alphabet. An application has an outcome, hit or miss, because the fallback operator mentions the miss. The specification

$$(A \mid B) \Rightarrow (C \to D)$$

says: apply A or B, then apply C, and apply D only if C missed. An apply block that applies D on the hit branch of C has a trace the grammar does not contain. An apply block that omits D entirely is still inside the grammar, because a hit of C is a complete trace. That asymmetry is deliberate. It is what the original rejected program shows, and it is not the same statement as "D must run when C misses."

Two readings are easy to confuse with this one, and both are worse fits. Equivalence would require the implementation to realize every word, so deleting the UDP arm would be an error. The note never rejects a missing arm. A temporal "eventually" reading of $\Rightarrow$ would allow unrelated tables between A and C. Nothing in the examples needs that extra room, and the header operator & is plainly concatenation. We use concatenation for both.

The grammar does not contain the expression that chooses IPv4 rather than IPv6. A checker that deleted an arm because it could not prove the arm feasible would be unsound: a feasible offending arm might be the one deleted. The checker therefore keeps every arm. It may alarm on an arm no packet takes. It does not accept a feasible arm outside the grammar.

3 Specification languages

3.1 Header words

Let H range over header type names and N over nonterminal names. Specifications are given by

$$\begin{array}{rcl} P & ::= & H \\ & | & P \,\&\, P \\ & | & P \mid P \\ & | & N. \end{array}$$

The production separator is the bar at the left. The operator $\mid$ is alternation inside an expression. A binding environment Γ maps each nonterminal to one specification. We require the dependency graph of Γ to be acyclic. The language $\mathcal{L}(P)$ is a finite set of words over header names.

$$\begin{aligned} \mathcal{L}(H) &= \{\, H \,\}, \\ \mathcal{L}(P_1 \,\&\, P_2) &= \{\, uv \mid u \in \mathcal{L}(P_1),\ v \in \mathcal{L}(P_2) \,\}, \\ \mathcal{L}(P_1 \mid P_2) &= \mathcal{L}(P_1) \cup \mathcal{L}(P_2), \\ \mathcal{L}(N) &= \mathcal{L}(\Gamma(N)). \end{aligned}$$

Acyclicity is what keeps the definition well-founded. A recursive nonterminal would describe a header stack. P4 has stacks, and a repetition operator is the honest way to add them. We do not add it in this fragment, because the source proposal has no example of one and a silent treatment of loops would change the claim.

A word w *validates* the set of names that occur in it. Names H_1 and H_2 are *exclusive* under P when no word of $\mathcal{L}(P)$ contains both.

▶ **Example 1.** $\mathcal{L}(\mathit{Ether} \,\&\, (\mathit{IPV4} \mid \mathit{IPV6}) \,\&\, (\mathit{TCP} \mid \mathit{UDP}))$ has four words. $\mathit{IPV4}$ and $\mathit{IPV6}$ are exclusive. Ether and TCP are not.

3.2 Hit/miss traces

Let t range over table names. An event is $\mathrm{hit}(t)$ or $\mathrm{miss}(t)$. Table specifications are given by

$$\begin{array}{rcl} T & ::= & t \\ & | & T \Rightarrow T \\ & | & T \to T \\ & | & T \mid T. \end{array}$$

For a language X, write $\mathrm{Hits}(X)$ for the traces in X whose last event is a hit, and $\mathrm{Misses}(X)$ for those whose last event is a miss.

$$\begin{aligned} \mathcal{L}(t) &= \{\, \mathrm{hit}(t),\ \mathrm{miss}(t) \,\}, \\ \mathcal{L}(T_1 \Rightarrow T_2) &= \{\, uv \mid u \in \mathcal{L}(T_1),\ v \in \mathcal{L}(T_2) \,\}, \\ \mathcal{L}(T_1 \mid T_2) &= \mathcal{L}(T_1) \cup \mathcal{L}(T_2), \\ \mathcal{L}(T_1 \to T_2) &= \mathrm{Hits}(\mathcal{L}(T_1)) \cup \{\, uv \mid u \in \mathrm{Misses}(\mathcal{L}(T_1)),\ v \in \mathcal{L}(T_2) \,\}. \end{aligned}$$

An atom contributes both outcomes, because a table that is merely applied can hit or miss. The miss-guard keeps every hitting trace of its left operand unchanged, and it appends the right operand only to missing traces. Sequencing does not inspect outcomes except insofar as they sit at the end of a trace and a surrounding miss-guard looks at them.

▶ **Example 2.** Let $T = (A \mid B) \Rightarrow (C \rightarrow D)$. The trace $\mathrm{hit}(A)\,\mathrm{hit}(C)$ is in $\mathcal{L}(T)$. The trace $\mathrm{hit}(A)\,\mathrm{miss}(C)\,\mathrm{hit}(D)$ is in $\mathcal{L}(T)$. The trace $\mathrm{hit}(A)\,\mathrm{hit}(C)\,\mathrm{hit}(D)$ is not. The trace $\mathrm{hit}(A)\,\mathrm{hit}(B)\,\mathrm{hit}(C)$ is not, because A and B are alternatives rather than a sequence.

The operator $\rightarrow$ is not syntactic sugar for a boolean expression inside P4. It constrains traces. A programmer can still write the test `t.apply().hit` in the implementation. Conformance then asks whether the traces of that test sit inside $\mathcal{L}(T)$.

▶ **Proposition 3.** *For every header specification and every table specification in the fragment, the language is finite.*

Proof. Header languages are built from singletons by union and concatenation, under an acyclic environment. Table languages are built the same way, with Hits and Misses selecting subsets. No constructor adds a Kleene star. ◀

4 From programs to traces

We do not repeat the semantics of P4 [14, 4]. We fix a fragment that is large enough for the examples and small enough that a path language is obviously finite.

4.1 Parsers

A parser fragment is a rooted directed graph. Vertices are parser states. An edge is labeled either by a header name, meaning `extract`, or by ε. The translation of the fragment is:

- `extract<H>` on the way into a state, or at the start of a state before the transition, contributes an H-labeled edge.
- `transition select` contributes one ε edge per arm. The arm's destination carries whatever that state extracts. The select expression itself is discarded.
- `transition accept` and `transition reject` end a path. Only `accept` produces a word.
- A call to a sub-parser is the graph of that sub-parser, inlined. The fragment has no recursive parser calls.

A word is the sequence of header labels along a path from the start to an accepting state, with ε erased. $\mathcal{L}(G)$ is the set of such words. If the graph contains a cycle, extraction fails and reports that the program is outside the fragment. It does not unroll the cycle a fixed number of times. A fixed unrolling would be an under-approximation and could hide a forbidden header on a later iteration.

Discarding select keys enlarges $\mathcal{L}(G)$ relative to the set of words a real packet can produce. That is the over-approximation discussed in Section 2.

4.2 Apply blocks

An apply fragment is a rooted directed graph whose labeled edges are $\mathrm{hit}(t)$, $\mathrm{miss}(t)$, or ε.

- `t.apply()` contributes a vertex with two outgoing edges, $\mathrm{hit}(t)$ and $\mathrm{miss}(t)$. If the result is tested, the hit edge enters the true branch and the miss edge enters the false branch, or the opposite when the test is negated. If the result is ignored, both edges enter the same continuation.
- A conditional that applies no table is an ε edge to each branch.
- Table applications that occur inside actions, and control-flow that reaches another control through a recursive call, are outside the fragment.

$\mathcal{L}(G)$ is the set of hit/miss words on paths from the start to a syntactic exit of the apply block. Cycles are rejected rather than unrolled, for the same reason as in parsers.

This graph says nothing about whether a hit is possible for the installed table entries. Control-plane contents are a different specification, already treated by annotation languages such as p4v [11]. Our alphabet records that the program asked the table and then took a branch. It does not record whether the control plane can populate a key that takes that branch.

5 Conformance

▶ **Definition 4.** *A parser graph G conforms to a header specification P when extraction succeeds and $\mathcal{L}(G) \subseteq \mathcal{L}(P)$. An apply graph G conforms to a table specification T when extraction succeeds and $\mathcal{L}(G) \subseteq \mathcal{L}(T)$.*

Inclusion is decidable on the fragment because both languages are finite. A depth-first enumeration of acyclic paths computes $\mathcal{L}(G)$. Membership of a word in $\mathcal{L}(P)$ or $\mathcal{L}(T)$ follows the equations of Section 3. The cost is linear in the number of paths, which is independent of header widths and of table sizes, and exponential in the worst case in the number of alternations. The specifications in the source proposal are short. The procedure is the one a compiler can run without a solver. It is not a procedure we have implemented.

▶ **Theorem 5.** *Assume extraction succeeds, and assume $\mathcal{L}(G)$ contains every trace the fragment can produce on a concrete packet. If G conforms to the specification, then every concrete trace is in the specification language.*

Proof. Conformance is $\mathcal{L}(G) \subseteq \mathcal{L}(spec)$. Every concrete trace is an element of $\mathcal{L}(G)$ by the assumption. The assumption holds for the fragment because every `select` arm and every hit/miss branch is materialized as an edge, and because constructs that could hide a trace are rejected at extraction time rather than dropped. ◀

Theorem 5 does not say the converse. A program can fail conformance because of a dead arm. That false alarm is the price of not consulting select keys or table entries. Theorem 5 also does not say that every word of the specification occurs in the program. Deleting a specified alternative preserves inclusion.

▶ **Proposition 6.** *If G' is obtained from a conforming graph G by deleting edges, and extraction still succeeds on G', then G' conforms to the same specification.*

Proof. Deleting edges shrinks or preserves the path language. A subset of a subset of the specification is still a subset. ◀

Proposition 6 is why the analysis is a filter on forbidden structure and not a test that the implementation realizes the architecture in full. A programmer who wants equivalence can ask for it as a separate check, $\mathcal{L}(G) = \mathcal{L}(spec)$. We do not take that as the default, because it rejects programs the original examples accept.

6 Header validity

P4 header instances are invalid until extracted, and a read of a field of an invalid header is a runtime error caught by architectures that enforce validity [14]. The grammatical specification gives a stronger static fact for the fragment: which combinations of headers may be valid together if the parser conforms.

▶ **Definition 7.** *Suppose a parser conforms to P. The headers that may be valid at the start of the ingress control are*

$$V(P) = \{\, \mathrm{names}(w) \mid w \in \mathcal{L}(P) \,\}.$$

A field read of H is justified when every set in $V(P)$ contains H, or when it is dominated by a validity test the fragment does not attempt to model. A pair of unguarded reads of H_1 and H_2 is unjustified when no set in $V(P)$ contains both.

▶ **Example 8.** Under Example 1, no set in $V(P)$ contains both *IPV4* and *IPV6*. An ingress block that binds `hdr.ipv4.src` and `hdr.ipv6.src` on the same straight-line path is unjustified. A block that tests `hdr.ipv4.isValid()` before the IPv4 read is outside the reasoning of Definition 7. The definition does not claim to replace P4's validity bits. It claims that, after a successful inclusion check, the grammar is a sound upper bound on the combinations those bits can exhibit.

The bound is only as good as conformance. If the parser graph is not a subset of $\mathcal{L}(P)$, a packet can validate a combination $V(P)$ never mentions, and a use-site check against $V(P)$ can accept a bad read or reject a good one. The use-site check therefore runs only after inclusion succeeds. This dependency is easy to miss if the grammar is treated as a comment.

7 Skeletons

The source note appeals to correctness-by-construction, but every program in it is written first and checked second. The semantics supports a limited constructing direction.

Define a translation $[\![P]\!]$ from an acyclic header specification to a parser graph with holes:

- $[\![H]\!]$ is a state that extracts H and accepts.
- $[\![P_1 \,\&\, P_2]\!]$ is the concatenation of the two graphs: every accept state of the first is wired to the start of the second, and those states no longer accept.
- $[\![P_1 \mid P_2]\!]$ is a fresh start state with two hole-edges, one into each translation. A hole-edge is an ε edge marked as an obligation to become a `select` arm.
- $[\![N]\!] = [\![\Gamma(N)]\!]$.

▶ **Theorem 9.** *Interpreting hole-edges as nondeterministic ε edges, $\mathcal{L}([\![P]\!]) = \mathcal{L}(P)$.*

Proof. By induction on P. A name extracts one header. Concatenation of graphs concatenates languages, matching the equation for &. A hole-start is the union of the two branches, matching alternation. Nonterminals follow the environment, which is acyclic, so the induction is well-founded. ◀

A filling of the holes replaces each hole-edge by one or more `select` arms. Any such replacement deletes the nondeterministic choices that the filler does not mention, or keeps them. It does not add a header extraction that the skeleton did not already contain, unless the programmer edits a state after generation. For fillings that only refine holes:

▶ **Corollary 10.** *If G is a filling of $[\![P]\!]$, then $\mathcal{L}(G) \subseteq \mathcal{L}(P)$.*

Proof. A filling's path language is a subset of the nondeterministic language. Theorem 9 equates that language with $\mathcal{L}(P)$. ◀

Corollary 10 is the construct-then-preserve statement the workshop note does not make. It is also easy to over-read. The corollary says nothing about whether the select expression

distinguishes IPv4 from IPv6 correctly. Two arms that both extract IPv4 still have a legal language. The packet format can be wrong in the bits and right in the grammar. Bit-level header typing remains the subject of SafeP4 and its dependently typed successor [6, 7]. The skeleton is a control skeleton, not a format checker.

We do not define a skeleton for table specifications. An apply block's Boolean conditions are rarely determined by the trace grammar alone. Emitting

```
if (A.apply().hit) {} else { B.apply(); }
```

from $A \mid B$ would invent a policy about which table is the primary. The grammar says the traces of A and of B are both legal and that a trace containing both is not. Several programs have exactly those traces. Choosing one of them is a programming step, not a consequence of $\mathcal{L}(T)$.

8 Case studies

The studies below are hand executions. They use the examples of the workshop note [17]. They are not runs of a tool, and they are not a sample of bugs in a corpus.

8.1 A forbidden parser arm

Take P from Example 1 and a parser whose start state extracts *Ether* and then selects either a state that extracts *IPV4* or a state that extracts *TCP*. Enumeration produces the words *Ether IPV4* and *Ether TCP*, among any further suffixes those states extract. *Ether TCP* is not in $\mathcal{L}(P)$. Definition 4 fails. The same graph with the TCP arm removed from the Ethernet state conforms only if every remaining word is one of the four. A later state that extracts TCP after IPv4 is fine. A state that extracts TCP after Ethernet is not. This is the distinction drawn by the comment in the workshop note, stated as membership.

8.2 Sub-parsers

Bind $L3 \mapsto IPV4 \mid IPV6$ and likewise for $L2$ and $L4$, and let the top specification be $L2 \,\&\, L3 \,\&\, L4$. Inlining three sub-parser calls in that order yields words in $\mathcal{L}(P)$, provided each sub-parser conforms to its own binding. Inlining $L4$ immediately after $L2$ yields words that skip $L3$. Those words fail membership. The nonterminal is doing real work: the check of the top parser does not need to see inside a sub-parser that has already been checked, as long as inlining uses the sub-parser's language rather than an unchecked body. A modular checker can cache $\mathcal{L}(\Gamma(N))$. We have not implemented the cache. The equations allow it.

8.3 Exclusive headers

Continuing from the conforming parser, $V(P)$ contains $\{Ether, IPV4, TCP\}$ and the three sibling sets, and no set contains both IP versions. The straight-line reads of `hdr.ipv4.src` and `hdr.ipv6.src` are unjustified by Definition 7. If the programmer inserts `isValid` tests, the definition does not apply, and this paper does not claim a result. That limit is preferable to a use-site analysis that pretends not to see the tests.

8.4 A fallback on the wrong branch

Take T from Example 2 and an apply block that applies A or B, then applies D when C hits. One enumerated trace is $\mathrm{hit}(A)\,\mathrm{hit}(C)\,\mathrm{hit}(D)$, which Example 2 excludes. The block

that applies D only on the miss of C enumerates traces that the example includes, together with traces that end at hit(C). All of those are in $\mathcal{L}(T)$. The block that applies A and then B before C is rejected, even though both tables occur "at the start" in an informal sense. Alternation is union, not a multiset of opening tables.

8.5 What these studies do not show

They do not show that the three table operators cover the pipelines in the Princeton Cabernet programs cited as motivation [17]. They do not show how often a dead `select` arm would raise a false alarm. They do not show that a p4c front end can build the graphs without dropping a `lookahead` or a parser exception. Those are empirical questions. The workshop note left them open, and this paper leaves them open.

9 Comparison with p4c

The reference compiler [13] implements P4. It does not implement the grammatical specification. We read the front end and the parser-unrolling mid-end at commit `98fbc63` (2 November 2026). p4c was not executed on the examples in this paper. The claims below are about what those passes do to a program of the shapes in Section 8.

9.1 What the model gets right

Four decisions in the model are the ones a compiler check would have to copy if it were to reject the same programs for the same reason.

- Conformance is inclusion of traces, not equivalence. An extra parser arm is an error. A missing alternative is not. That is the distinction drawn by the TCP-after-Ethernet example and by Proposition 6.
- The miss-guard $T_1 \to T_2$ admits the continuation only on a miss. hit(C) followed by hit(D) is outside $\mathcal{L}(C \to D)$. A hit of C with no following D is inside it (Example 2).
- After a parser conforms, header validity is the set of names in the accepted word. A straight-line read of two exclusive headers is unjustified (Definition 7). Calling `isValid` is not, by itself, evidence that a header was extracted.
- Every `select` arm is kept, and a parser cycle is rejected rather than unrolled a guessed number of times. Dropping an arm would be an under-approximation and could hide a feasible forbidden word. Unrolling a cycle whose bound is unknown is the same kind of under-approximation.

9.2 Architectural programs the compiler accepts

No front-end pass builds a language of header words or of hit/miss traces. `SimplifyParsers` deletes states that cannot reach `accept` or `reject`, and it errors only when neither is reachable. A reachable arm that extracts TCP immediately after Ethernet is kept. `CheckCoreMethods` checks that `extract` is applied to a header. It does not check which header may follow which. The word *Ether TCP* therefore survives compilation and fails Definition 4.

`tableApply` desugars `apply().hit`, `apply().miss`, and `apply().action_run`. It does not constrain the order of applications. An apply block that runs D on the hit branch of C type-checks. The trace hit(A) hit(C) hit(D) is outside Example 2.

Co-occurrence is not computed. `SimplifyDefUse` tracks each header validity bit on its own, with values yes, no, and maybe. A missing fact is maybe, and maybe produces

`WARN_INVALID_HEADER`, which does not stop compilation. Two maybe-headers are not an error. An unguarded read of both IPv4 and IPv6 fields is a legal P4 program and an unjustified use under Definition 7.

These three programs are legal P4. Accepting them is correct compilation of the language. It is the wrong outcome for the architecture the grammar states. The model is the check that has the architecture. p4c does not.

9.3 Where the compiler's own warning goes wrong

`SimplifyDefUse` is the pass that comes closest to header validity, and it is unsound for the warning it emits.

`HeaderDefinitions::find` returns yes when the header is in a set called `notReport`, without consulting the validity bit. A call to `isValid` inserts the receiver into that set. The comment on the set says that `isValid` in a condition is not interpreted, so every header mentioned by the call is stored until the end of the block. The visitor for `if` evaluates the condition first and only then visits both branches. Both branches therefore see the header as not worth reporting. After the `if`, the set is restored to its value from before the condition.

The false branch of `if (hdr.ipv4.isValid())` can read `hdr.ipv4.srcAddr` with no `WARN_INVALID_HEADER`. So can the true branch of the negated test. The pass never looks at the polarity. A discarded call, `hdr.ipv4.isValid();` followed by a field read in the same block, suppresses the warning until `setValid` or `setInvalid`, which are the updates that erase the set. The warning claims to report "accessing a field of an invalid header" and then does not report that access.

`extract` is not one of those updates. On an extern method, including `packet_in.extract`, the same pass skips the arguments. The comment says that extern methods are assumed to set output headers valid. The code continues before any such assignment. The operation that makes a header valid in P4 is invisible to the warning. `setValid` is visible. The warning is therefore not a model of the parser.

Definition 7 does not have this hole. A field read is justified only when every specification word that can reach the read contains the header. Mentioning `isValid` does not add the header to that set.

9.4 Where unrolling can go wrong

`ParserUnroll` symbolically executes parser loops so that header-stack indexes become constants. When `checkLoops` cannot count the iterations it warns `WARN_INVALID` and sets an error flag. The caller then clears the flag and continues with the remaining states. The program is not rejected. The cycle is not faithfully unrolled either.

The model refuses the same program. The grammar has concatenation and union only. A cycle is outside the fragment. Guessing a bound, or continuing after admitting that no bound was found, can drop a later iteration that would have extracted a forbidden header. That is the under-approximation Section 4 refuses.

9.5 What this comparison does not show

We did not compile the examples, and we did not count how often the warning fires on a corpus. Backend miscompilations, of the kind Gauntlet searches for [15], are outside the grammar: a correct architectural check does not catch a bad code generator. The agreements are small and real. `RemoveParserControlFlow` keeps both branches of a parser `if`, which is

Table 1 What existing analyses decide, against the inclusion question of this paper.

Analysis	Question	Needs
p4c [13]	Is the program well-typed P4?	The P4 text
p4v [11]	Do assertions about headers, tables, and state hold?	Control-plane annotations, solver
Vera [16]	Do symbolic SEFL paths violate checks?	A translation of the program
ASSERT-P4 [8]	Do symbolic C paths violate assertions?	Assertions in the program
SafeP4 [6], dependent types [7]	Are header and mask operations well typed?	A type derivation
Leapfrog [5]	Are two parsers equivalent?	A second parser, a proof
Gauntlet [15]	Does the compiler miscompile?	Generated programs
This paper	Is every fragment trace in the grammar?	A short grammar

the same over-approximation as keeping every `select` arm. Unreachable states are deleted, and inclusion allows a missing alternative.

10 Related work

Programmable data planes sit at the end of two lines of work: checking a fixed network, and checking the program that defines the network. Table 1 keeps only the second line, and only the question each system decides. The rest of this section says what each system actually consumes, and why a header grammar is not one of those inputs.

10.1 The language and its compiler

P4 separates parsing from match-action control and makes both programmable [2, 14]. The language specification fixes the meaning of `extract`, `select`, header validity, and `apply().hit`. It does not fix which header sequences a given program is supposed to accept, and it does not fix the order in which that program's tables are supposed to run. Those are properties of one program, not of the language. The BNF in the P4 specification is the concrete syntax of parser declarations. It is not a grammar of packet headers. The metalanguage in Section 3 re-uses concatenation and alternation for the packet language. It is not a proposal to change P4's grammar.

The reference compiler [13] is the implementation of that language. Its front end type-checks, rejects an `extract` whose argument is not a header, deletes unreachable parser states, and turns parser conditionals into `select`. Its mid-end unrolls parser loops when it can count them. All of that is necessary for a correct compilation, and none of it is a specification of the architecture. Section 9 records the consequences. A reachable arm that extracts TCP immediately after Ethernet is well-typed P4, so p4c keeps it. An apply block that runs a fallback table on a hit is well-typed P4, so p4c keeps it. The invalid-header warning inside p4c is the one analysis that looks like Definition 7, and Section 9 shows why it is not: `isValid` suppresses the warning on both branches, and `extract` is not treated as the update that makes a header valid. p4c is the compiler a grammatical checker would have to extend. It is not a substitute for the checker.

10.2 Assertion checkers

Three systems developed in the same year translate a P4 program into another language and search for violations. They can express the examples in this paper, because a forbidden header sequence is a reachable state and a forbidden table order is a control-flow path. They ask the programmer for a different artifact than a grammar, and they discharge it with a solver or a symbolic executor rather than with inclusion of a finite word language.

p4v translates parsers, controls, tables, and actions into a guarded-command language and checks assertions, including safety properties that mention header validity [11]. Control-plane annotations constrain which table entries may exist. That is a strictly richer specification than ours: it talks about values and about entries, and it needs an SMT solver. A grammar does not mention entries. It also does not need a solver on the acyclic fragment, because both languages are finite (Proposition 3). We do not claim that enumeration scales to the programs p4v already verifies. We claim that the specifications in the workshop note are the size of a grammar, not the size of an assertion proof.

Vera translates P4 into the symbolic language SEFL and executes paths to find bugs [16]. The translation is aimed at debugging a program that is already written. There is no separate description of the legal header words or of the legal hit/miss traces. A bug is whatever the symbolic encoding was instrumented to notice. Our conformance relation is the other way around. The grammar is the specification, and a path is a bug only when its word falls outside that language. Vera remains the better tool when the property is not a trace language, for example a relationship between two header fields that depends on table entries.

ASSERT-P4 takes assertions written against the P4 program, translates the annotated program to C, and symbolically executes the resulting paths [8]. Parser states and table actions are in scope. The assertion is local: it is a predicate at a program point, such as “these two headers are not both valid here.” Definition 7 derives that same fact from the set of words, after inclusion has succeeded, and it derives it for every point rather than for the points the programmer annotated. ASSERT-P4 can state properties our grammar cannot, including arithmetic constraints on fields. It cannot state the pipeline $(A \mid B) \Rightarrow (C \to D)$ as one expression whose language is the allowed set of traces. Encoding that pipeline as assertions is possible and is exactly the encoding we are trying not to ask for.

The three checkers also differ in what they trust. p4v, Vera, and ASSERT-P4 trust a translation into guarded commands, SEFL, or C. Leapfrog later observed that none of those translations is proved against a reusable semantics of P4 [5]. Our extraction has the same kind of debt, stated as the assumption of Theorem 5. The difference is the size of the translation. On the fragment, extraction is the parser graph and the apply graph of Section 4, not a model of actions and table entries.

10.3 Types for header layout

SafeP4, in this venue, gives a type system for header bits and masks so that a concatenation or a slice cannot silently drop or invent a field [6]. The later dependently typed calculus refines that account so that the type of a value can depend on a header that has already been parsed [7]. Both systems answer a question we do not ask: whether the bits inside a header are manipulated safely. Our grammars answer whether the sequence of headers, and the sequence of tables, is the intended one. A program can be well typed in SafeP4 and still extract TCP immediately after Ethernet, if both header types are legal types. It can satisfy our grammar and still mis-handle a checksum. The checks compose. Neither type system has a miss-guard on tables. Table order is control structure, and SafeP4 leaves control structure

to the programmer.

10.4 Semantics and parser equivalence

Petr4 supplies a definitional interpreter for P4 and a core calculus with a proof of type-preserving termination [4]. It is the semantic account a mechanized version of Theorem 5 should be stated against. Petr4 does not take a programmer-written grammar of header words or of table traces. Our extraction is not a function on Petr4's syntax. Until it is, the theorem is conditional on Section 4.

Leapfrog proves equivalence of two protocol parsers [5]. The model is automata. The proof is a bisimulation, discharged with SMT for the bit-vector side conditions, and the metatheory is in Coq. Equivalence is the right theorem when a reference parser already exists. It is stronger than inclusion, and it is the wrong shape of specification when the reference is a three-line grammar rather than a second implementation. A grammar is easier to write than a reference parser and weaker as a definition of the protocol: it says which header names occur and in which order, not which bits select the arm. Leapfrog's observation that p4v, Vera, and ASSERT-P4 verify one program against assertions, rather than a relation between two parsers, is the same cut we use. We add a third point on that cut. The specification can be a language of traces, checked by inclusion, without being either an assertion set or a second parser.

10.5 Compiler bugs

Gauntlet searches for crash bugs and semantic bugs in compilers for programmable packet processing, including p4c [15]. A grammatical specification is a property of the source program, not of the code generator. The two meet only when a compiler drops a `select` arm or reorders tables. That would be a miscompilation relative to P4, which is Gauntlet's question, and a conformance failure relative to the grammar, which is ours. We do not use one analysis to test the other. Section 9 is not a Gauntlet-style bug hunt. It is a reading of passes that already run, and of the programs those passes accept.

10.6 Where this paper starts

Fixed-function network verification checks reachability and forwarding invariants of a configured network. Header-space analysis does this geometrically on the space of packet headers [9]. NetKAT gives the forwarding policy a Kleene algebra with tests and a decision procedure [1]. Both assume the parser and the tables as a fixed box. P4 opens that box. The properties in this paper are inside the box: which headers the parser may produce, and in which order the tables may run. They are not network-wide reachability.

The workshop note proposes to write those properties as BNF-like annotations and to check them in the compiler [17]. It cites correctness-by-construction [3, 12, 10] and leaves the definitions and the implementation open. Correctness-by-construction derives a program from a specification so that conformance holds by construction. Section 7 is the part of this paper that does that, and only for parser control, up to holes for `select`. The inclusion check is not correctness-by-construction. It is a check of a program that was written first, which is what the examples in the note actually do. The contribution relative to the note is the trace semantics, the statement that conformance is inclusion, the reading of the miss-guard, and the account of why p4c, Vera, and ASSERT-P4 do not already decide that inclusion.

11 Limitations

The fragment excludes parser loops, header stacks, `lookahead`, `verify`, parser value sets, tables applied from actions, and recursive control calls. A compiler that skipped those constructs and checked the rest would make Theorem 5 false. The required engineering behaviour is to refuse the program or to confine the claim to the subgraph that was actually extracted and to say so in the diagnostic.

Select keys and table entries are ignored. False alarms are possible. False acceptance of a feasible forbidden arm is not, inside the fragment.

Inclusion does not require the implementation to cover the specification. Programmers who want that guarantee need a second mode. We have not defined diagnostics for it beyond equality of the two finite languages.

Header-use checking does not understand `isValid`, assignments that invalidate a header, or deparsing. It is a corollary of a conforming parser, not a replacement for architectural validity checks.

No implementation of the checker in p4c exists. The workshop note listed that implementation as future work, and it remains future work. Section 9 is a reading of the passes p4c does run. It is not a count of alarms or of bugs on a corpus.

A mechanized proof would interpret the fragment inside Petr4 and prove Theorem 5 there. That would also fix the extraction, which is the least formal part of this paper: it is a list of translation rules rather than a function defined on Petr4's syntax.

12 Conclusion

A grammatical specification of header sequences and table pipelines is small enough to sit next to a P4 program and precise enough to define. On an acyclic fragment, conformance is inclusion of an over-approximating path language in a finite trace language, the miss-guard means "only if missed," and a parser skeleton matches the grammar up to holes for `select`. Those statements are what the workshop proposal needs before a compiler pass can be honest about what it checks. The reference compiler accepts the extra parser arm and the fallback-on-hit pipeline, and its invalid-header warning is suppressed by a mention of `isValid`, including on the branch where the header is invalid. They do not subsume solver-based verification or header type systems. They describe the architectural check those systems were not built to be.

References

1 Carolyn Jane Anderson, Nate Foster, Arjun Guha, Jean-Baptiste Jeannin, Dexter Kozen, Cole Schlesinger, and David Walker. NetKAT: Semantic foundations for networks. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL, pages 113–126, 2014. doi:10.1145/2535838.2535862.

2 Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.*, 44(3):87–95, 2014. doi:10.1145/2656877.2656890.

3 Edsger W. Dijkstra. *A Discipline of Programming*. Prentice Hall, 1976.

4 Ryan Doenges, Mina Tahmasbi Arashloo, Santiago Bautista, Alexander Chang, Newton Ni, Samwise Parkinson, Rudy Peterson, Alaia Solko-Breslin, Amanda Xu, and Nate Foster. Petr4: Formal foundations for P4 data planes. In *Proc. ACM Program. Lang.*, volume 5, 2021. doi:10.1145/3434322.

5 Ryan Doenges, Tobias Kappé, John Sarracino, Nate Foster, and Greg Morrisett. Leapfrog: Certified equivalence for protocol parsers. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI, 2022. doi:10.1145/3519939.3523715.

6 Matthias Eichholz, Eric Campbell, Nate Foster, Guido Salvaneschi, and Mira Mezini. How to avoid making a billion-dollar mistake: Type-safe data plane programming with SafeP4. In *33rd European Conference on Object-Oriented Programming (ECOOP 2019)*, volume 134 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 12:1–12:28, 2019. doi:10.4230/LIPIcs.ECOOP.2019.12.

7 Matthias Eichholz, Eric Hayden Campbell, Matthias Krebs, Nate Foster, and Mira Mezini. Dependently typed data plane programming. *Proc. ACM Program. Lang.*, 6(POPL), 2022. doi:10.1145/3498701.

8 Lucas Freire, Miguel Neves, Lucas Leal, Kirill Levchenko, Alberto Schaeffer-Filho, and Marinho Barcellos. Uncovering bugs in P4 programs with assertion-based verification. In *Proceedings of the Symposium on SDN Research*, SOSR, 2018. doi:10.1145/3185467.3185499.

9 Peyman Kazemian, George Varghese, and Nick McKeown. Header space analysis: Static checking for networks. In *9th USENIX Symposium on Networked Systems Design and Implementation*, NSDI, 2012. URL: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/kazemian.

10 Derrick G. Kourie and Bruce W. Watson. *The Correctness-by-Construction Approach to Programming*. Springer, 2012.

11 Jed Liu, William Hallahan, Cole Schlesinger, Milad Sharif, Jeongkeun Lee, Robert Soulé, Han Wang, Călin Caşcaval, Nick McKeown, and Nate Foster. p4v: Practical verification for programmable data planes. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM, pages 490–503, 2018. doi:10.1145/3230543.3230582.

12 Carroll Morgan. *Programming from Specifications*. Prentice Hall, 2 edition, 1994.

13 P4.org Architecture Working Group. P4 reference compiler (P4C). https://github.com/p4lang/p4c, 2016.

14 P4.org Language Consortium. $P4_{16}$ language specification. https://p4.org/p4-spec/docs/P4-16-spec.html, 2024. Accessed 2 November 2026.

15 Fabian Ruffy, Tao Wang, and Anirudh Sivaraman. Gauntlet: Finding bugs in compilers for programmable packet processing. In *14th USENIX Symposium on Operating Systems Design and Implementation*, OSDI, 2020. URL: https://www.usenix.org/conference/osdi20/presentation/ruffy.

16 Radu Stoenescu, Dragos Dumitrescu, Matei Popovici, Lorina Negreanu, and Costin Raiciu. Debugging P4 programs with Vera. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM, pages 518–532, 2018. doi:10.1145/3230543.3230548.

17 Keyu Yuan and Baber Rehman. Formal specifications for data plane programs. In *Proceedings of the 2nd Workshop on Formal Methods Aided Network Operation*, pages 49–51, 2025. doi:10.1145/3750022.3750458.